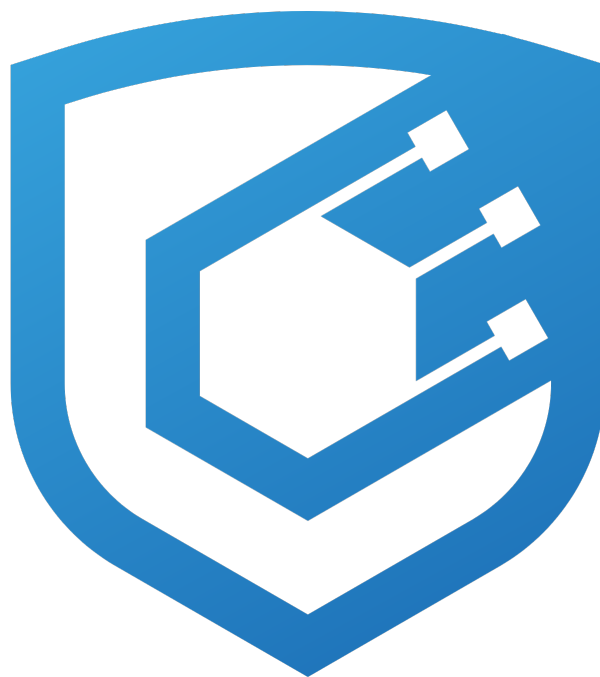




CoBuilders Cache Manager Automation Audit Report

Prepared by [Cyfrin](#)
Version 2.0

Lead Auditors
[Dacian](#)
[SBSecurity](#) ([Blckhv](#), [Slavcheww](#))

August 31, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	3
6.1	Centralization Risks	4
6.2	Design Decisions	4
6.2.1	Permissionless CacheManagerAutomation::placeBids	4
6.2.2	No nonReentrant modifier on CacheManagerAutomation::placeBids	4
7	Findings	7
7.1	Low Risk	7
7.1.1	CacheManagerAutomation::_shouldBid calls getMinBid without try/catch	7
7.1.2	.env.example hard-codes Arbitrum Sepolia CacheManager as ARB_ONE_CACHE_MANAGER_-ADDRESS resulting in bricked deployment	8
7.1.3	CacheManagerAutomation::insertContract does not enforce minFundAmount or maxUserFunds	8
7.1.4	CacheManagerAutomation::placeBids can re-bid the same contract within one batch via mid-batch eviction	9
7.1.5	CacheManagerAutomation::updateContract accepts _maxBid below minMaxBidAmount	9
7.1.6	Owner-only setters in CacheManagerAutomation lack meaningful bounds and cross-validation	10
7.1.7	Programs disabled by a Stylus version upgrade are never re-activated by the automation	11
7.1.8	A shared codehash pins a user's cache slot to a co-tenant's bid, so maxBid cannot be enforced	12
7.2	Informational	15
7.2.1	hardhat.config.ts does not pin evmVersion explicitly	15
7.2.2	CacheManagerAutomation::constructor performs insufficient validation of external addresses	15
7.2.3	Prefer Ownable2Step to Ownable	16
7.2.4	Unused {Escrow} import in BiddingEscrow	16
7.2.5	CacheManagerAutomation::_calculateBidAmount adds an unnecessary bid-uniqueness term	17
7.2.6	CacheManagerAutomation::insertContract, updateContract events omit the enabled flag	18
7.2.7	BiddingEscrow::withdrawForBid emits Withdrawn(depositor, amount) but funds go to owner	18
7.2.8	Misleading dead upgrade configuration for a non-upgradeable contract	18
7.2.9	Automation re-activates expired programs instead of using the cheaper keepalive	19
7.2.10	Use named mapping parameters to make the purpose of keys and values explicit	20
7.2.11	Remove obsolete return statement when already using named returns	20
7.2.12	Don't initialize to default values in Solidity	21
7.3	Gas Optimization	22
7.3.1	CacheManagerAutomation::_shouldBid runs precompile reads before cheap registration checks	22
7.3.2	Use custom errors instead of string revert reasons	22
7.3.3	Pack ContractConfig address and bools into one storage slot	23
7.3.4	Use calldata instead of memory for read-only internal struct parameters	23
7.3.5	Right-size and pack owner-configurable storage parameters	24
7.3.6	Use msg.sender inside onlyOwner functions instead of owner() to optimize away unnecessary storage read	25
7.3.7	Use ReentrancyGuardTransient for faster nonReentrant modifiers	25
7.3.8	Emit events first to optimize away old values only used for event emission	25

7.3.9 Use named return variables when this can optimize away a local variable 26

7.3.10 Cache identical storage reads 26

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Cache Manager Automation system is an operator-driven helper around the Arbitrum Stylus cache-slot bidding mechanism, also automating reactivation of expired programs and programs requiring a Stylus version upgrade. Users register their Stylus WASM contracts using `CacheManagerAutomation`, fund per-user escrows held in the child `BiddingEscrow`, and the system places bids against the upstream Arbitrum `CacheManager` contract so the user's program stays warm in the cache. Activation and cache-status reads go through the `ArbWasm 0x71` and `ArbWasmCache 0x72` precompiles.

These three external dependencies (`CacheManager`, `ArbWasm`, `ArbWasmCache`) are distinct contract addresses supplied separately to the constructor. The owner controls only configuration parameters (bid increments, cache thresholds, horizon windows, per-user limits); user funds are held in a dedicated `BiddingEscrow` whose ownership is permanently the automation contract, so the deployer cannot directly withdraw user balances.

5 Audit Scope

The audit scope was limited to:

```
contracts/core/BiddingEscrow.sol
contracts/core/CacheManagerAutomation.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Cache Manager Automation](#) code provided by [CoBuilders](#). The findings consist of 8 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The owner role of `CacheManagerAutomation`:

- controls a set of parameters which govern the functioning of `CacheManagerAutomation`, finding L-6 implemented tighter constraints in setter functions to prevent values which could negatively impact users
- cannot directly withdraw user deposited funds from the `BiddingEscrow` because the escrow's `onlyOwner` modifier resolves to the `CacheManagerAutomation` contract, not the deployer EOA

Both of contracts `BiddingEscrow` and `CacheManagerAutomation` are immutable and don't have any pause or upgrade functionality.

6.2 Design Decisions

The protocol has a number of explicit design decisions; we note the most important ones here together with potential negative consequences.

6.2.1 Permissionless `CacheManagerAutomation::placeBids`

We didn't find any realistic exploit scenarios since:

- the bid is always `_calculateBidAmount` output which is capped at user's max bid
- it can only target the user's own registered & enabled contracts, funded from their own escrow
- a successful bid caches the user's own contract - the exact service they registered and funded for; a failed bid is re-deposited

However we did find and note some potential consequences of this design:

- theoretically the same contract could be successfully bid multiple times in one transaction due to eviction mid-batch. In practice this doesn't seem exploitable however the mitigation is simple and defensive so should be done; at most one successful non-zero bid per user and codehash within an Arbitrum L2 block (fixed in finding L-4)
- sub-optimal spending is possible as a third-party may trigger bids at a timing and frequency that would be less optimal than what a user themselves would choose; this seems like a reasonable trade-off for the design choice since the user still caps their max bid and contracts which bids can be spent on. If this does become problematic in the future, a solution could require users to explicitly allow-list third-party operators
- caller-controlled batch order controls pricing within the batch - within the same batch contracts processed while utilization is below `cacheThreshold` cache for free. If earlier bids push utilization across `cacheThreshold`, the remaining contracts in the same batch must compete and pay. `placeBids` being permissionless means the caller can choose that order

The last item gives the caller an advantage but this seems like a reasonable design trade-off as:

- earlier bids genuinely fill the cache, so later bids correctly see less slack and pay to compete
- the "harm" is bounded by the victim's own `maxBid`, the bid caches the victim's contract (the service they funded), and the victim pays the legitimate full-cache rate within their authorization
- hence it falls into the "sub-optimal spending" category which seems like a reasonable design trade-off for allowing the function to be permissionless

`CacheManagerAutomation::placeActivations` is also permissionless and we didn't find any exploit scenarios.

6.2.2 No `nonReentrant` modifier on `CacheManagerAutomation::placeBids`

This appears safe as the only external calls made during `placeBids` don't allow the possibility of re-entry: `CacheManager::getMinBid`, `cacheSize`, `queueSize`, `decay`, `placeBid`, `BiddingEscrow::depositsOf`, `withdrawForAutomation`, `deposit`, `ArbWasmCache::codehashIsCached`, `ArbSys::arbBlockNumber`.

Summary

Project Name	Cache Manager Automation
Repository	stylus-cm-contracts
Commit	5a4a62000928...
Fix Commit	82f963ae4544...
Audit Timeline	May 21st - May 27th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	8
Informational	12
Gas Optimizations	10
Total Issues	30

Summary of Findings

[L-1] <code>CacheManagerAutomation::_shouldBid</code> calls <code>getMinBid</code> without <code>try/catch</code>	Resolved
[L-2] <code>.env.example</code> hard-codes Arbitrum Sepolia CacheManager as <code>ARB_ONE_CACHE_MANAGER_ADDRESS</code> resulting in bricked deployment	Resolved
[L-3] <code>CacheManagerAutomation::insertContract</code> does not enforce <code>minFundAmount</code> or <code>maxUserFunds</code>	Resolved
[L-4] <code>CacheManagerAutomation::placeBids</code> can re-bid the same contract within one batch via mid-batch eviction	Resolved
[L-5] <code>CacheManagerAutomation::updateContract</code> accepts <code>_maxBid</code> below <code>minMaxBidAmount</code>	Resolved
[L-6] Owner-only setters in <code>CacheManagerAutomation</code> lack meaningful bounds and cross-validation	Resolved
[L-7] Programs disabled by a Stylus version upgrade are never re-activated by the automation	Resolved
[L-8] A shared codehash pins a user's cache slot to a co-tenant's bid, so <code>maxBid</code> cannot be enforced	Acknowledged
[I-01] <code>hardhat.config.ts</code> does not pin <code>evmVersion</code> explicitly	Resolved
[I-02] <code>CacheManagerAutomation::constructor</code> performs insufficient validation of external addresses	Acknowledged
[I-03] Prefer <code>Ownable2Step</code> to <code>Ownable</code>	Resolved

[I-04] Unused Escrow import in BiddingEscrow	Resolved
[I-05] CacheManagerAutomation::_calculateBidAmount adds an unnecessary bid-uniqueness term	Acknowledged
[I-06] CacheManagerAutomation::insertContract, updateContract events omit the enabled flag	Resolved
[I-07] BiddingEscrow::withdrawForBid emits Withdrawn(depositor, amount) but funds go to owner	Resolved
[I-08] Misleading dead upgrade configuration for a non-upgradeable contract	Resolved
[I-09] Automation re-activates expired programs instead of using the cheaper keepalive	Acknowledged
[I-10] Use named mapping parameters to make the purpose of keys and values explicit	Resolved
[I-11] Remove obsolete return statement when already using named returns	Resolved
[I-12] Don't initialize to default values in Solidity	Acknowledged
[G-1] CacheManagerAutomation::_shouldBid runs precompile reads before cheap registration checks	Resolved
[G-2] Use custom errors instead of string revert reasons	Resolved
[G-3] Pack ContractConfig address and bools into one storage slot	Resolved
[G-4] Use calldata instead of memory for read-only internal struct parameters	Resolved
[G-5] Right-size and pack owner-configurable storage parameters	Acknowledged
[G-6] Use msg.sender inside onlyOwner functions instead of owner() to optimize away unnecessary storage read	Resolved
[G-7] Use ReentrancyGuardTransient for faster nonReentrant modifiers	Acknowledged
[G-8] Emit events first to optimize away old values only used for event emission	Acknowledged
[G-9] Use named return variables when this can optimize away a local variable	Resolved
[G-10] Cache identical storage reads	Resolved

7 Findings

7.1 Low Risk

7.1.1 CacheManagerAutomation::_shouldBid calls getMinBid without try/catch

Description: CacheManagerAutomation::_shouldBid calls getMinBid on the external cacheManager at contracts/core/CacheManagerAutomation.sol:600 with no try/catch, unlike the sibling reads in _calculateBidAmount and _shouldActivate which are all wrapped. Because _shouldBid runs inside the placeBids loop with no isolation, one reverting entry reverts the whole batch, including every unrelated legitimate bid.

CacheManager::getMinBid resolves the program codehash through ArbWasm::codehashAsmSize → getActiveProgram, and reverts on four distinct inputs, none of which _shouldBid checks:

- AsmTooLarge - the program asm size is larger than cacheSize;
- ProgramNotActivatedError - the codehash is not an activated Stylus program (a plain EOA, a regular non-Stylus contract, a never-activated Stylus contract, or a codeless address whose codehash resolves to the empty hash);
- ProgramExpiredError - the activation has aged past ExpiryDays;
- ProgramNeedsUpgradeError - the program version no longer matches the current Stylus version.

<https://github.com/OffchainLabs/nitro/blob/6dce8d1/arbos/programs/programs.go#L536-L557>

The Solidity 0.8 extcodesize guard does not catch the ProgramNotActivated case: that guard checks the call target cacheManager (which has code), not the contractAddress argument.

An entry can reach this revert two ways. The first is benign: a legitimately registered Stylus program that expires while uncached. The second is a **zero-capital, permissionless attack**. insertContract rejects only address(0) (// TODO allow on only stylus contracts at contracts/core/CacheManagerAutomation.sol:220) and never checks that _contract is an activated Stylus program. Registration also requires no funds: _maxBid is just a stored config number checked against minMaxBidAmount, and the deposit is msg.value, which can be 0 because the insertContract deposit path skips the minFundAmount floor that fundBalance enforces (see issue 4). So an attacker calls:

```
insertContract(poisonAddr, minMaxBidAmount, /*_enabled=*/true, false, 0); // msg.value = 0
```

paying only gas, where poisonAddr is any EOA or non-activated address. The entry passes every _shouldBid gate - it is not the zero address, it is not cached, it is enabled, and it is owned by the registrant - so the function reaches getMinBid and reverts. (A disabled entry never reaches getMinBid, since _shouldBid returns at the enabled check first.)

Impact: Any registered, enabled entry whose getMinBid reverts blocks every placeBids batch that includes it. No funds are directly at risk: withdrawBalance does not touch cacheManager, so user balances stay withdrawable.

The disruption is not limited to an honest operator's naive batch, because the poison entry is cheap to plant and hard to filter:

- **Free to plant.** Any unprivileged actor can register a reverting entry for the cost of gas alone, with no escrow deposit (see the msg.value = 0 path above), and can repeat cheaply with many entries or simply leave them in place at no maintenance cost.
- **No on-chain signal distinguishes poison from healthy.** Operators build batches from the on-chain registry (getContracts), where a poison entry is indistinguishable from a healthy enabled, not-cached contract. Filtering requires per-item simulation of every entry every cycle.
- **Time-sensitive.** Bids matter because the cache evicts continuously, so a stalled batch can let healthy users' contracts get evicted before the operator detects and excludes the poison entry.

This is a cheap, permissionless denial of service against the protocol's core flow (placeBids).

Recommended Mitigation: Primary - isolate the failure on-chain. Wrap getMinBid in try/catch like the sibling precompile reads, returning a skip result on revert so one bad entry no longer collapses the batch:

```
uint192 minBid;
try cacheManager.getMinBid(contractAddress) returns (uint192 m) {
    minBid = m;
} catch {
    return BidResult(false, contracts[j], 0);
}
```

Defense-in-depth - validate at registration. Complete the `insertContract` TODO and reject addresses that are not activated Stylus programs (e.g. require a successful `cacheManager.getMinBid(_contract)` or an `ArbWasm` activation check at insert time), so the poison entry never enters the registry in the first place.

CoBuilders: Fixed in commit [0e8f2ed](#).

Cyfrin: Verified.

7.1.2 `.env.example` hard-codes Arbitrum Sepolia CacheManager as `ARB_ONE_CACHE_MANAGER_ADDRESS` resulting in bricked deployment

Description: `.env.example:18` ships `ARB_ONE_CACHE_MANAGER_ADDRESS=0x0c9043d042ab52cfa8d0207459260040`

That literal is the Arbitrum Sepolia CacheManager, not the Arbitrum One CacheManager. Line 11 of the same file uses the identical value for `ARB_SEPOLIA_CACHE_MANAGER_ADDRESS`, so the committed example file labels one address as belonging to two different chains. `config/constants.ts` resolves `CACHE_MANAGER_ADDRESSES::arbitrumOne` from `process.env.ARB_ONE_CACHE_MANAGER_ADDRESS` with the env override winning, so any deployer who follows the documented `cp .env.example .env` plus `npm run deploy:arbitrum` flow wires the CMA constructor to a codeless mainnet address. The canonical Arbitrum One CacheManager is `0x51dedbd2f190e0696afbee5e60bfde96d86464ec`.

Impact: A documented-setup Arbitrum One mainnet deployment is bricked: `placeBids` will revert on every call because the wired `cacheManager` has no code on mainnet. `cacheManager` is immutable in `CacheManagerAutomation` and there is no setter, so the only recovery is a full redeploy plus user-side re-funding. Because `insertContract` and `fundBalance` do not touch `cacheManager`, users may fund the dead contract before the operator notices that `placeBids` reverts; those funds must then be withdrawn and re-deposited into the redeployed instance.

Recommended Mitigation: Correct `.env.example:18` to the canonical Arbitrum One CacheManager address:

```
ARB_ONE_CACHE_MANAGER_ADDRESS=0x51dedbd2f190e0696afbee5e60bfde96d86464ec
```

Add a defense-in-depth chain-id pin to `config/constants.ts` so that resolving the Arbitrum One CacheManager from an env override is rejected if the resolved value does not match the canonical mainnet address.

CoBuilders: Fixed in commit [a80e15d](#).

Cyfrin: Verified.

7.1.3 `CacheManagerAutomation::insertContract` does not enforce `minFundAmount` or `maxUserFunds`

Description: `CacheManagerAutomation::insertContract` is payable and forwards `msg.value` to `_updateUserBalance` at `contracts/core/CacheManagerAutomation.sol:255` with no deposit checks. The sibling `fundBalance` (`contracts/core/CacheManagerAutomation.sol:346-354`) enforces both a floor (`msg.value >= minFundAmount`) and a ceiling (`escrow.depositsOf(msg.sender) + msg.value <= maxUserFunds`). `insertContract` enforces neither, so a user can deposit below the floor or above the per-user ceiling through this path. The `maxUserFunds` cap is therefore enforced on one funding entry point and not the other.

Impact: Limited. The deposit is keyed to `msg.sender`, so a user can only over- or under-fund their own escrow, and the balance stays fully withdrawable via `withdrawBalance`. Escrow balances are per-user isolated, so exceeding `maxUserFunds` does not affect other users or protocol solvency. This is a missing-check asymmetry between the two funding paths, with no third-party fund loss.

Recommended Mitigation: Apply the ceiling check `fundBalance` uses, and apply the floor only when a deposit is actually made so that registering a contract without funding stays possible:

```
if (msg.value > 0 && msg.value < minFundAmount) revert InvalidFundAmount();
if (escrow.depositsOf(msg.sender) + msg.value > maxUserFunds)
    revert ExceedsMaxUserFunds();
```

CoBuilders: Fixed in commit [7c760a5](#).

Cyfrin: Verified.

7.1.4 CacheManagerAutomation::placeBids can re-bid the same contract within one batch via mid-batch eviction

Description: CacheManagerAutomation::_shouldBid re-reads arbWasmCache.codehashIsCached(contractAddress.codehash) on every iteration of the placeBids loop, and placeBids does not deduplicate contractAddress (or contractAddress.codehash) within a batch. When a bid succeeds, the upstream CacheManager::placeBid caches the program, but if a later entry in the same batch bids a competing contract that evicts it, the cached flag flips back to false, so a subsequent entry for the original contract bids again. A batch of alternating entries [{u, A}, {u, B}, {u, A}, {u, B}, ...] can therefore bid the same contracts repeatedly in one call instead of once each, each successful bid debiting the user's escrow.

Impact: In theory a caller could repeat alternating entries up to maxBidsPerIteration so a user's escrow is debited many times in one placeBids call (churning the same cache slot) rather than once per contract, draining far more than the intended one bid per contract.

In practice this is not exploitable on the deployment targets, for two independent reasons:

1. Bids are free unless the cache is near full. _calculateBidAmount returns 0 (a no-cost bid) while cacheUtilization < cacheThreshold (default 98%). The Arbitrum One cache is cacheSize = 536870912 (512 MB) and only about 22% used (queueSize around 119997440), so bids currently debit nothing. The global Stylus cache would have to grow roughly 4x before any paid bidding occurs.
2. Even at full utilization, a 512 MB cache holds about 766 programs of this size, so a bid evicts the single globally-cheapest entry, which is some other program rather than the victim's partner contract. Forcing two of the victim's own contracts to evict each other requires the cache to fit roughly one program, which is not the real configuration and is not attacker-controllable. The victim's bids also escalate by decay * horizonSeconds each round, so they stop being the cheapest entries and any incidental repeat self-limits to one or two iterations.

So the realistic worst case is one or two extra bids under an already-full cache, not a multi-bid drain. It remains a correctness bug worth fixing.

Recommended Mitigation: Enforce at most one successful non-zero bid per (user, codehash, Arbitrum L2 Block).

CoBuilders: Fixed in commit [67a1362](#).

Cyfrin: Verified.

7.1.5 CacheManagerAutomation::updateContract accepts _maxBid below minMaxBidAmount

Description: CacheManagerAutomation::insertContract at line 221 enforces if (_maxBid < minMaxBidAmount) revert InvalidBid();. The sibling CacheManagerAutomation::updateContract at lines 265-300 omits the same check and overwrites contracts[i].maxBid with _maxBid unconditionally. The protocol design invariant maxBid >= minMaxBidAmount (intended to bound the per-bid magnitude) is enforced on one entry point and ignored on the other.

Impact: A user can register at the floor and immediately lower their maxBid below minMaxBidAmount (down to 0 since _maxBid is a plain uint256 and there is no > 0 check either).

Recommended Mitigation: Add the floor check matching insertContract:

```
function updateContract(
    address _contract,
    uint256 _maxBid,
```

```

    bool _enabled,
    bool _autoActivate,
    uint256 _maxActivationCost
) external {
    if (_contract == address(0)) revert InvalidAddress();
    if (_maxBid < minMaxBidAmount) revert InvalidBid();
    // ...
}

```

CoBuilders: Fixed in commit [98171f8](#).

Cyfrin: Verified.

7.1.6 Owner-only setters in CacheManagerAutomation lack meaningful bounds and cross-validation

Description: Several owner-only setters in CacheManagerAutomation accept values that defeat the contract's own logic, documentation, or cross-parameter invariants, with no meaningful bounds or cross-validation:

- `setCacheThreshold` accepts any value ≤ 100 , including 0 (no > 0 floor). With `cacheThreshold = 0`, the `cacheUtilization < cacheThreshold` guard in `_calculateBidAmount` is never true, so every bid takes the paid branch even when the cache has free space and `getMinBid` returns 0. Users then pay `decayValue` (capped at their `maxBid`) to cache, non-refundably, when caching would otherwise be free.
- `setHorizonSeconds` and `setBidIncrement` accept arbitrarily large values (only > 0). `_calculateBidAmount` computes `decayValue = minBid + decayRate * horizonSeconds + bidIndex * bidIncrement` in checked `uint256` arithmetic, so an extreme value overflows and reverts, collapsing the whole `placeBids` batch.
- `setMaxUsersPerPage` accepts any value > 0 with no upper bound, despite the consumer `getContractsPaginated` documenting a hard cap of 100 (NatSpec at line 443: capped at 100; comment at line 461: max 100 users per call). The effective cap is whatever the owner last set, so the code contradicts its own documentation. Setting it large defeats the pagination safety cap: a single `getContractsPaginated` call with `limit == 0` (or `limit` above the cap) then attempts to return every remaining user at once and can run out of gas.
- `setMinFundAmount` accepts any value > 0 and is not cross-checked against `maxUserFunds`. Setting `minFundAmount > maxUserFunds` makes `fundBalance` permanently revert for top-ups: any `msg.value >= minFundAmount` necessarily also breaches the `currentBalance + msg.value > maxUserFunds` ceiling.
- `setMaxUserFunds` lowers the per-user cap (only > 0) without reconciling existing `ContractConfig` entries. Configs whose `maxActivationCost` exceeds the new cap dead-end auto-activation — `_shouldActivate` requires `cfg.maxActivationCost <= escrow.depositsOf(user)`, and the user can no longer fund that high — until the owner raises the cap again or the user lowers `maxActivationCost` via `updateContract`.

Impact: All of these require the owner (a trusted role) to set an adverse value, are recoverable by re-tuning the parameter (or, for `setMaxUserFunds`, by users re-running `updateContract`), and cannot be triggered by an external party — owner misconfiguration / missing validation, not an attacker-reachable path. `setCacheThreshold(0)` makes users overpay for caching they could have had for free; an extreme `setHorizonSeconds` / `setBidIncrement` makes `placeBids` revert (a self-inflicted operator DoS), bounded by each user's `maxBid`; `setMinFundAmount > maxUserFunds` bricks `fundBalance` top-ups; lowering `setMaxUserFunds` pauses auto-activation for grandfathered configs. `setMaxUsersPerPage` only bounds a view getter with no on-chain consumer, so its worst case is a failing off-chain read. No fund loss in any case.

Recommended Mitigation: Bound and cross-validate the setters against the protocol's operating envelope (illustrative values):

```

require(_cacheThreshold > 0 && _cacheThreshold <= 100, 'cacheThreshold out of range');
require(_horizonSeconds > 0 && _horizonSeconds <= 365 days, 'horizonSeconds out of range');
require(_bidIncrement > 0 && _bidIncrement <= 1 ether, 'bidIncrement out of range');
require(_maxUsersPerPage > 0 && _maxUsersPerPage <= 100, 'maxUsersPerPage out of range');
require(_minFundAmount > 0 && _minFundAmount <= maxUserFunds, 'minFundAmount out of range');

```

For `setMaxUserFunds`, either document the cap as forward-only (existing configs keep their original `maxActivationCost` until re-tuned) or sweep existing configs and clamp/flag any `maxActivationCost` above the new cap.

CoBuilders: Fixed in commit [5fce1ff](#).

Cyfrin: Verified noting:

- `setMaxUserFunds` chose the forward-only option
- max limit also added to `setMaxBidsPerIteration`

7.1.7 Programs disabled by a Stylus version upgrade are never re-activated by the automation

Description: `_shouldActivate` decides whether the automation re-activates a user's registered program. It calls `programTimeLeft` and treats the program as activatable only when the call returns 0 or reverts with `ProgramExpired`. Any other revert is treated as skip.

<https://github.com/CoBuilders-xyz/stylus-cm-contracts/blob/5a4a6200928de3b86e7a7e6313ba54946d6912b/contracts/core/CacheManagerAutomation.sol#L730-L741>

```
try arbWasm.programTimeLeft(contractAddress) returns (uint64 timeLeft) {
    if (timeLeft != 0) return ActivationResult(false, cfg);
} catch (bytes memory revertData) {
    bytes4 sel;
    if (revertData.length >= 4) {
        assembly {
            sel := mload(add(revertData, 32))
        }
    }
    if (sel != PROGRAM_EXPIRED_SELECTOR)
        return ActivationResult(false, cfg);
}
```

On Arbitrum a Stylus version upgrade bumps the global `params.Version`. After that, every program activated under the previous version no longer matches the current version. The precompile `programTimeLeft` calls `getActiveProgram`, which checks the version before it checks expiry, so for these programs it reverts with `ProgramNeedsUpgrade`, not `ProgramExpired`.

<https://github.com/OffchainLabs/nitro/blob/master/arbos/programs/programs.go#L546-L555>

```
// check that the program is up to date
stylusVersion := params.Version
if program.version != stylusVersion {
    return program, ProgramNeedsUpgradeError(program.version, stylusVersion)
}

// ensure the program hasn't expired
if program.ageSeconds > arbmth.DaysToSeconds(params.ExpiryDays) {
    return program, ProgramExpiredError(program.ageSeconds)
}
```

`ProgramNeedsUpgrade` has selector `0x637d968f`, which is not `PROGRAM_EXPIRED_SELECTOR` (`0xc9b12e52`), so `_shouldActivate` returns false. The automation skips the program even though `activateProgram` is the correct fix and the user opted in by setting `autoActivate`.

Impact: After a Stylus version upgrade, programs the user registered for auto activation stay disabled and uncallable. The automation silently skips them, so the user must re-activate manually, which defeats the purpose of registering them. No funds are at risk and version upgrades are rare chain wide events, so the severity is low. This is a missed liveness guarantee.

Recommended Mitigation: Treat `ProgramNeedsUpgrade` as an activation trigger alongside `ProgramExpired`, since `activateProgram` resolves both. Add the selector as a constant and accept it in the catch block.

```
// bytes4(keccak256("ProgramNeedsUpgrade(uint16,uint16)"))
bytes4 private constant PROGRAM_NEEDS_UPGRADE_SELECTOR = 0x637d968f;
```

```
if (sel != PROGRAM_EXPIRED_SELECTOR && sel != PROGRAM_NEEDS_UPGRADE_SELECTOR)
    return ActivationResult(false, cfg);
```

Keep the design as an allowlist of activation triggers rather than activating on any revert that is not expired. Other reverts such as a low level read error or ProgramNotActivated should still skip unless first activation of registered programs is an intended feature. Also update the comment at lines 725 to 729, which states expiry is the only revert path and understates the cases programTimeLeft can revert with.

CoBuilders: Fixed in commit [d9005e4](#).

Cyfrin: Verified.

7.1.8 A shared codehash pins a user's cache slot to a co-tenant's bid, so maxBid cannot be enforced

Description: The Arbitrum cache is keyed by program codehash, not by address. Two contracts deployed from identical bytecode share one codehash, so they share one cache slot.

_shouldBid skips a contract as soon as its codehash is cached, without looking at the bid that backs the slot.

<https://github.com/CoBuilders-xyz/stylus-cm-contracts/blob/5a4a620/contracts/core/CacheManagerAutomation.sol#L585-L587>

```
// Is contract already cached?
if (arbWasmCache.codehashIsCached(contractAddress.codehash))
    return BidResult(false, ContractConfig(address(0), 0, false, false, 0), 0);
```

A user sets maxBid to express how hard they want their slot defended against eviction. But the upstream CacheManager.placeBid reverts AlreadyCached when the codehash is already cached, and offers no way to raise the bid on an existing entry.

<https://github.com/OffchainLabs/nitro/blob/6dce8d1/contracts/src/chain/CacheManager.sol#L196-L209>

```
function placeBid(
    address program
) external payable {
    if (isPaused) {
        revert BidsArePaused();
    }
    bytes32 codehash = program.codehash;
    if (_isCached(codehash)) {
        revert AlreadyCached(codehash);
    }
    ...
}
```

So whoever caches the codehash first sets the bid that backs the slot, and maxBid is unreachable for everyone else who shares that bytecode. The first bidder can be an attacker who deploys the same bytecode, or just another honest user with a low maxBid. Because placeBids is permissionless and processes one request at a time, the slot stays pinned to that first low bid. The automation never bids again for the shared codehash, so it cannot move the slot up toward the higher user's maxBid.

The same codehash keying applies to activation. programTimeLeft and activateProgram resolve the address to its codehash and read program state by codehash, so an address reads as activated once any twin is activated, and _shouldActivate then skips it. This is a related free riding effect on activation cost. The pin on maxBid is the part with no on chain remedy and is the focus of this report.

Impact: A user who funds escrow and sets a high maxBid to keep their contract cached can be silently downgraded to a co-tenant's lower bid. The slot is then first in line for eviction, so the contract drops out of the cache earlier

than the user's `maxBid` would have allowed, and calls to it pay the higher uncached cost until the next cycle. No funds are lost, since escrow stays withdrawable and the user is never overcharged, they are under protected.

This only bites in the paid regime, when cache utilization is at or above `cacheThreshold` (default 98). While the global Stylus cache has free space, `_calculateBidAmount` returns 0 for everyone, so low and high users both bid 0 and the pin has no economic effect.

There is no on chain fix inside the automation. `CacheManager` has no bid top up for a cached codehash, and only the chain owner can evict, so no automation could enforce `maxBid` for a shared codehash. Severity Low.

Proof of Concept: The test deploys the unmodified `CacheManagerAutomation` and `BiddingEscrow` against a faithful `CacheManager` mock that reproduces the three relevant Nitro behaviors: the cache is keyed by codehash, `placeBid` reverts `AlreadyCached` for a cached codehash, and a full cache evicts the lowest bid first. Two deployments of an identical `DummyProgram` share a real EVM codehash. `cacheThreshold` is set to 0 to put the automation in the paid regime.

Test 1 shows the pin. The low user is processed first and the slot is backed by 2 wei. The high user, funded with 1 ether of `maxBid`, is skipped by `_shouldBid` with no escrow debit, and a direct fully funded `placeBid` for the high user's own address reverts `AlreadyCached`.

Test 2 shows the harm. The pinned slot at bid 2 is evicted by an unrelated bid of 3, while a bid of 5 survives, so the high user's contract is dropped by a bid the high user could easily have beaten.

```
function testCodehashPin_lowBidderPinsHighBidder() public {
    cm.setCacheSize(10); // plenty of room; isolate the pin from eviction

    // Two DISTINCT addresses, IDENTICAL bytecode => SAME codehash.
    DummyProgram pLow = new DummyProgram();
    DummyProgram pHigh = new DummyProgram();
    assertEq(address(pLow).codehash, address(pHigh).codehash, 'shared codehash');
    bytes32 codehash = address(pLow).codehash;

    uint256 lowMaxBid = 2; // wei (capped tiny)
    uint256 highMaxBid = 1 ether; // willing to pay a lot to defend the slot

    vm.prank(userLow);
    cma.insertContract{value: 1 ether}(address(pLow), lowMaxBid, true, false, 0);
    vm.prank(userHigh);
    cma.insertContract{value: 1 ether}(address(pHigh), highMaxBid, true, false, 0);

    // Step 1: backend processes the LOW user first.
    cma.placeBids(_bidReq(userLow, address(pLow)));
    assertTrue(awc.codehashIsCached(codehash));
    assertEq(cm.bidForCodehash(codehash), uint192(lowMaxBid)); // slot backed by 2 wei

    // Step 2: backend processes the HIGH user.
    // _shouldBid sees codehashIsCached==true, returns shouldBid=false.
    uint256 highBalBefore = escrow.depositsOf(userHigh);
    cma.placeBids(_bidReq(userHigh, address(pHigh)));
    assertEq(escrow.depositsOf(userHigh), highBalBefore); // escrow untouched
    assertEq(cm.bidForCodehash(codehash), uint192(lowMaxBid)); // still pinned at 2 wei

    // Step 3: no on chain top up path.
    vm.expectRevert(
        abi.encodeWithSelector(FaithfulCacheManager.AlreadyCached.selector, codehash)
    );
    cm.placeBid{value: 1 ether}(address(pHigh));
}

function testCodehashPin_pinnedSlotEvictsEarly() public {
    DummyProgram pLow = new DummyProgram();
    DummyProgram pHigh = new DummyProgram();
    bytes32 codehash = address(pLow).codehash;
```

```

// Slot A: unrelated program cached at a healthy bid of 5.
DummyOther filler = new DummyOther();
cm.placeBid{value: 5}(address(filler));

// Slot B: victim codehash, pinned at low bid 2.
vm.prank(userLow);
cma.insertContract{value: 1 ether}(address(pLow), 2, true, false, 0);
vm.prank(userHigh);
cma.insertContract{value: 1 ether}(address(pHigh), 1 ether, true, false, 0);
cma.placeBids(_bidReq(userLow, address(pLow)));
assertEq(cm.bidForCodehash(codehash), 2);

// A new program enters the full cache with bid 3.
// _makeSpace evicts the lowest entry = victim (bid 2), not the filler (5).
DummyThird newcomer = new DummyThird();
cm.placeBid{value: 3}(address(newcomer));

assertFalse(awc.codehashIsCached(codehash)); // victim evicted
assertTrue(awc.codehashIsCached(address(filler).codehash)); // bid-5 filler survived
}

```

Result:

```

[PASS] testCodehashPin_lowBidderPinsHighBidder()
[PASS] testCodehashPin_pinnedSlotEvictsEarly()

```

Recommended Mitigation: This cannot be fully fixed on chain because CacheManager has no bid top up for a cached codehash. Recommended steps:

1. Document the limitation for users. A contract that shares bytecode with another deployed contract may be cached at a bid lower than its `maxBid`, set by whoever bid first, and the automation cannot raise it. Users who need a defended slot should deploy unique bytecode.
2. Have the off chain backend read the live entry bid that backs a registered contract's `codehash` and surface it, so a user can see when their slot is pinned below their `maxBid`.
3. If a future CacheManager adds a bid increase path for a cached `codehash`, change `_shouldBid` to compare the current backing bid against the user's target and bid up when it is too low, instead of skipping on `codehashIsCached` alone.

CoBuilders: Acknowledged:

- Arbitrum's CacheManager identifies entries by codehash rather than contract address and does not expose an operation to increase or replace the bid once that codehash is cached. CacheManagerAutomation therefore cannot independently increase protection for a second address deploying identical bytecode
- No user funds are lost. A second contract sharing the codehash may receive less eviction protection than its configured `maxBid` would otherwise permit. We will consider exposing shared-codehash state to off-chain clients; automated top-ups require future upstream CacheManager support

7.2 Informational

7.2.1 `hardhat.config.ts` does not pin `evmVersion` explicitly

Description: `hardhat.config.ts` selects Solidity 0.8.30 with `viaIR` but does not set `evmVersion`, so the EVM target is whatever the toolchain defaults to. With the pinned Hardhat and solc versions that default resolves to `paris` (verified: `npx hardhat compile` reports `evm target: paris`), a conservative target that every chain in scope (Arbitrum One, Arbitrum Sepolia, Superposition) supports. So there is no current opcode-compatibility issue and no bytecode mismatch against the committed Ignition journals, which were also produced for `paris`.

The remaining concern is determinism: relying on the implicit default means a future Hardhat or solc upgrade could change the target (for example to `cancun` or `prague`) with no change to the project, yielding different bytecode across builds and across networks. Pinning the value makes the build reproducible and immune to toolchain default drift.

Recommended Mitigation: Pin `evmVersion` to the conservative target supported by every chain in scope:

```
solidity: {
  version: '0.8.30',
  settings: {
    optimizer: { enabled: true, runs: 200 },
    viaIR: true,
    evmVersion: 'paris',
  },
},
```

CoBuilders: Fixed in commit [0b60eb5](#).

Cyfrin: Verified.

7.2.2 `CacheManagerAutomation::constructor` performs insufficient validation of external addresses

Description: `CacheManagerAutomation::constructor` validates only that `_cacheManager`, `_arbWasmCache`, and `_arbWasm` are non-zero. It performs no further sanity checks, and all three are stored immutable with no setter, so any misconfiguration requires a redeploy to correct. Two gaps:

1. **No code-length check.** A deployment wired to a mistyped, wrong-chain, or codeless `cacheManager` is not caught at construction.
2. **No mutual-distinctness check.** Aliasing any pair (for example `_cacheManager == _arbWasm`) passes the constructor but produces an internally inconsistent contract.

This is a deployment-time hardening gap, not an attacker-reachable issue: with correct addresses the contract behaves correctly, and a misconfiguration is the deployer's to make and to detect. Detection is loud in both cases — the deploy script logs the resolved addresses, and the first `placeBids` (or activation) call surfaces a bad or aliased address as a selector-mismatch revert rather than silent misrouting. Funding and withdrawal still work, so no funds are lost; recovery is a redeploy.

Recommended Mitigation: Add a code-length check for the `cacheManager` address, which is the one that can realistically be misconfigured to a codeless value:

```
if (_cacheManager.code.length == 0) revert InvalidAddress();
```

Applying the same check to `_arbWasmCache` / `_arbWasm` is redundant on a real Arbitrum chain (the `0x71` / `0x72` precompiles always have code) and would make the constructor revert on a plain Hardhat network where those addresses are codeless, so scope the code check to `cacheManager` unless the test setup provides precompile code.

Additionally, require the three addresses to be mutually distinct (the code-length check above does not catch aliasing to a coded precompile):

```
if (_cacheManager == _arbWasmCache || _cacheManager == _arbWasm || _arbWasmCache == _arbWasm)
  revert InvalidAddress();
```

CoBuilders: Acknowledged:

- the constructor already rejects zero addresses for all three immutable dependencies. These addresses are selected through a controlled deployment workflow and cannot be modified after deployment
- code-length and pairwise-distinctness checks could detect some deployment mistakes earlier, but we classify them as deployment hardening rather than a runtime security boundary. They could also encode assumptions about Arbitrum system and precompile addresses that may not apply uniformly across supported networks
- a misconfigured dependency causes the corresponding operation to fail while user escrow remains withdrawable; recovery is redeployment. We retain the existing zero-address checks and controlled deployment verification

7.2.3 Prefer Ownable2Step to Ownable

Description: CacheManagerAutomation uses OpenZeppelin's single-step Ownable. transferOwnership has no acceptance step (a mistyped destination is unrecoverable), and renounceOwnership can permanently lock every onlyOwner setter on this non-upgradeable contract. Both are self-inflicted owner actions with no fund-loss path (withdrawBalance is user-facing and the bid/activation flows are permissionless), so the worst case is frozen admin parameters.

Recommended Mitigation: Inherit Ownable2Step instead of Ownable for two-step transfers, and override renounceOwnership to revert:

```
import {Ownable2Step} from '@openzeppelin/contracts/access/Ownable2Step.sol';

contract CacheManagerAutomation is ICacheManagerAutomation, Ownable2Step, ReentrancyGuard {
    function renounceOwnership() public override onlyOwner {
        revert('renounce disabled');
    }
    // ...
}
```

CoBuilders: Fixed in commit [86c8639](#).

Cyfrin: Verified.

7.2.4 Unused {Escrow} import in BiddingEscrow

Description: BiddingEscrow.sol:4 imports {Escrow} from @openzeppelin/contracts/utils/escrow/Escrow.sol, but the contract declares contract BiddingEscrow is Ownable (line 21) and never references Escrow — it reimplements the deposit/withdraw API in its own functions rather than inheriting. The import is dead code.

The same leftover scaffolding also left the file-level NatSpec (lines 8-20) as OpenZeppelin's Escrow documentation verbatim (@title Escrow, "Base escrow contract, holds funds designated for a payee..."), describing a base contract that BiddingEscrow does not inherit. This can mislead a reader about the contract's lineage, storage layout, and event shape.

This is the only unused import across the in-scope contracts (every import in CacheManagerAutomation is used).

Recommended Mitigation: Two options:

- 1) Remove the unused import, and update the file-level NatSpec to describe BiddingEscrow's own API rather than OpenZeppelin's Escrow:

```
// remove from BiddingEscrow.sol:
import {Escrow} from '@openzeppelin/contracts/utils/escrow/Escrow.sol';
```

- 2) Refactor the BiddingEscrow.sol contract to import from OZ Escrow. This is likely the best option since the only unique part of BiddingEscrow is the function withdrawForBid so there's no need for BiddingEscrow to duplicate all of the functionality already present in Escrow.

CoBuilders: Fixed in commit [da46dae](#).

Cyfrin: Verified.

7.2.5 CacheManagerAutomation::_calculateBidAmount adds an unnecessary bid-uniqueness term

Description: CacheManagerAutomation::_calculateBidAmount (lines 557-560, reached only in the cache-full branch) adds `bidIndex * bidIncrement` to the bid. The inline comment at line 555 states the intent: "Bid index * BID_INCREMENT is used to make the bid unique for all the contract bids in current block." `bidIndex` is the CacheManagerAutomation::placeBids loop counter: the position of the entry in the caller-supplied BidRequest array.

The term serves no purpose, and its stated justification is false:

1. CacheManager::placeBid does not require unique bids. CacheManager::_makeSpace accepts equal bids: it reverts only on `bid < min`, and its own comment notes "if the new bid equals to the minimum bid, a random entry with minimum bid will be evicted". The bid heap also tolerates duplicate values, so CacheManagerAutomation bids never need to be unique.
2. The term does not prevent within-batch self-eviction. If the cache is full and a user's just-cached entry is the current minimum, a later bid evicts it regardless: `_makeSpace` always pops the minimum entry, then accepts any bid greater than or equal to `min`. A `bidIncrement`-sized nudge does not change which entry is the minimum: that is governed by the decay buffer (`decayRate * horizonSeconds`), which dwarfs the increment. The genuine self-eviction concern is tracked separately and is fixed by per-batch contract de-duplication, not by this term.

Because `bidIndex` is the caller-controlled array position, the term also lets a caller choose which entry gets the largest increment. The magnitude is dust: at most $(\text{maxBidsPerIteration} - 1) * \text{bidIncrement}$, i.e. 49 wei at the default `bidIncrement` of 1, capped at the user's own `maxBid`, funded from the user's own escrow to cache the user's own contract, and only reachable when the cache is full (`_calculateBidAmount` returns 0 below `cacheThreshold`). There is no meaningful impact; the term is dead weight justified by a false premise.

Recommended Mitigation: Remove the `bidIndex * bidIncrement` term from `decayValue`:

```
uint256 decayValue = uint256(minBid) + (uint256(decayRate) * horizonSeconds);
```

That term is the only use of both `bidIndex` and `bidIncrement`, so the complete cleanup removes both:

1. `bidIncrement` and all its machinery: the CacheManagerAutomation::bidIncrement state variable (line 40), its constructor default (line 81), the CacheManagerAutomation::setBidIncrement setter (lines 185-189), and the setBidIncrement declaration (line 219) and BidIncrementUpdated event (line 137) in ICacheManagerAutomation
2. The `bidIndex` parameter threaded through CacheManagerAutomation::placeBids, `_shouldBid`, and `_calculateBidAmount` (drop both parameters and the `i` argument)

Equal bids across batch entries are accepted by CacheManager, so nothing else is required. Do not replace the term with a block-number or running-nonce scheme: an unbounded multiplier would push `decayValue` past `maxBid` and force every full-cache bid to the user's maximum.

CoBuilders: Acknowledged:

- we intentionally retain the existing `bidIndex * bidIncrement` component. It adds a deterministic batch-position component before the result is capped by the user's configured `maxBid`
- the increment is not required for CacheManager acceptance, does not guarantee globally unique bids, and is not a security control against eviction. Request ordering can influence the result, but the final amount remains bounded by each user's explicit `maxBid` authorization
- removing it would change established bidding behavior and remove its owner configuration surface without resolving a security vulnerability, so we retain it for this release

7.2.6 CacheManagerAutomation::insertContract, updateContract **events omit the enabled flag**

Description: CacheManagerAutomation::updateContract (lines 281-284) overwrites all four mutable Contract-Config fields (maxBid, enabled, autoActivate, maxActivationCost), but the emitted events never include enabled: ContractUpdated carries maxBid, ContractAutoActivateUpdated carries autoActivate, and ContractMaxActivationCostUpdated carries maxActivationCost. The new enabled value is set silently.

CacheManagerAutomation::insertContract has the same gap: it stores enabled in the new config (line 250) but emits only ContractAdded (carrying maxBid), ContractAutoActivateUpdated, and ContractMaxActivationCostUpdated (lines 256-262).

So enabled is never emitted on insert or update, and an event-only indexer cannot track it without querying getUserContracts. This is a meaningful field: enabled gates whether a contract is bid for in _shouldBid (line 596). There is no security impact; this is an event-completeness gap.

Recommended Mitigation: Emit enabled from both insertContract and updateContract, either by extending the existing ContractAdded / ContractUpdated events to carry it or by adding a parallel event:

```
event ContractEnabledUpdated(address indexed user, address indexed contractAddress, bool enabled);

// in insertContract and updateContract, alongside the existing events:
emit ContractEnabledUpdated(msg.sender, _contract, _enabled);
```

CoBuilders: Fixed in commit [f4307b8](#).

Cyfrin: Verified.

7.2.7 BiddingEscrow::withdrawForBid **emits Withdrawn(depositor, amount) but funds go to owner**

Description: BiddingEscrow::withdrawForBid at lines 79-94 emits Withdrawn(depositor, amount) at line 93, the same event shape that BiddingEscrow::withdraw emits at line 64. The withdraw payee receives the ETH; withdrawForBid sends the ETH to owner (the parent CacheManagerAutomation). An off-chain indexer subscribing to Withdrawn cannot distinguish the two cases and assumes the named depositor received the funds.

Recommended Mitigation: Emit a distinct event for the bid-withdrawal path so off-chain consumers can tell the two cases apart:

```
event WithdrawnForBid(address indexed depositor, address indexed recipient, uint256 weiAmount);

function withdrawForBid(address depositor, uint256 amount) public onlyOwner {
    // ... existing logic
    emit WithdrawnForBid(depositor, msg.sender, amount);
}
```

CoBuilders: Fixed in commit [6663754](#) noting:

- withdrawForBid was renamed to withdrawForAutomation, reflecting that it funds bidding and activation
- now emits WithdrawnForAutomation(address indexed depositor, address indexed recipient, uint256 amount), distinguishing automation spending from a user's own withdrawal and recording CacheManagerAutomation as recipient

Cyfrin: Verified.

7.2.8 Misleading dead upgrade configuration for a non-upgradeable contract

Description: The project documents a non-upgradeable architecture (README "Non-Upgradeable Design": "Direct contract deployment (no proxy patterns)", "Immutable contract logic once deployed"), and CacheManagerAutomation is deployed directly with no proxy (ignition/modules/CacheManagerAutomation.ts calls m.contract('CacheManagerAutomation', ...)). Yet the deployment configuration retains upgrade-related scaffolding that nothing consumes:

- `config/constants.ts`: `DEFAULT_CONFIG.upgradeDelay = 86400` (line 51) and `GAS_LIMITS.upgrade = 1000000` (line 58).
- `config/deployment-config.ts`: `upgradeDelay` in the `DeploymentConfig` interface (line 14) and set for every network (lines 25, 34, 43, 52, 61).
- `hardhat.config.ts`:4: `import '@openzeppelin/hardhat-upgrades'`; — the upgrades plugin is imported but never used (no `deployProxy` / `upgradeProxy` anywhere).

Impact: Informational. No functional or security impact — the values are inert dead config. But they are misleading: a maintainer or integrator reading the config reasonably infers a 24-hour upgrade timelock and an upgrade path that do not exist, directly contradicting the documented non-upgradeable design. The `@openzeppelin/hardhat-upgrades` import is also an unused dependency.

Recommended Mitigation: Remove the dead upgrade scaffolding to match the non-upgradeable architecture:

- delete `upgradeDelay` from `DEFAULT_CONFIG` (`constants.ts`) and from the `DeploymentConfig` interface + per-network entries (`deployment-config.ts`);
- delete `GAS_LIMITS.upgrade` (`constants.ts`);
- remove `import '@openzeppelin/hardhat-upgrades'`; from `hardhat.config.ts` (and drop the dependency from `package.json` if otherwise unused).

If upgradeability is in fact intended, implement it explicitly — but the README states the design is non-upgradeable.

CoBuilders: Fixed in commit [31f02e6](#).

Cyfrin: Verified.

7.2.9 Automation re-activates expired programs instead of using the cheaper keepalive

Description: A Stylus program has two costs to keep it usable. `activateProgram` runs the full WASM activation, which charges activation gas for the re-compile plus a data fee. `CodehashKeepalive` only extends the expiry date and charges the data fee, with no re-compile, so it is the cheaper way to keep an active program from lapsing.

`CodehashKeepalive` calls `ProgramKeepalive`, which calls `getActiveProgram` first. That reverts once the program has expired, so keepalive is only valid while the program is still active.

<https://github.com/OffchainLabs/nitro/blob/master/arbos/programs/programs.go#L581-L593>

```
func (p Programs) ProgramKeepalive(codeHash common.Hash, time uint64, params *StylusParams) (*big.Int,
↳ error) {
    program, err := p.getActiveProgram(codeHash, time, params)
    if err != nil {
        return nil, err
    }
    if program.ageSeconds < arbmeth.DaysToSeconds(params.KeepaliveDays) {
        return nil, ProgramKeepaliveTooSoon(program.ageSeconds)
    }
    ...
}
```

The automation never uses keepalive. Its only expiry related path is `_shouldActivate`, which returns true only once `programTimeLeft` returns 0 or reverts with `ProgramExpired`, that is, only once the program has already expired.

<https://github.com/CoBuilders-xyz/stylus-cm-contracts/blob/5a4a62000928de3b86e7a7e6313ba54946d6912b/contracts/core/CacheManagerAutomation.sol#L730-L741>

```
try arbWasm.programTimeLeft(contractAddress) returns (uint64 timeLeft) {
    if (timeLeft != 0) return ActivationResult(false, cfg);
} catch (bytes memory revertData) {
    bytes4 sel;
    if (revertData.length >= 4) {
        assembly {

```

```

        sel := mload(add(revertData, 32))
    }
}
if (sel != PROGRAM_EXPIRED_SELECTOR)
    return ActivationResult(false, cfg);
}

```

By the time this fires, keepalive would revert because the program is expired, so re-activation is the only remaining option. The trigger and the keepalive precondition never overlap. There is no near expiry path that would call keepalive while the program is still active and `ageSeconds` is at least `KeepaliveDays`.

Impact: For programs the user wants kept activated, the automation waits until expiry and pays the full activation cost, when a keepalive before expiry would have extended the program for only the data fee. Users registered with `autoActivate` pay more than necessary over time. This is a cost inefficiency, not a loss of funds or a safety issue.

Note that keepalive does not affect caching. A kept alive program can still be evicted from the cache, which is the contract's main concern, so keepalive only optimizes the activation lifecycle.

Recommended Mitigation: If minimizing the cost of keeping programs activated is a goal, add a proactive keepalive path that fires while the program is still active. Read the remaining time from `programTimeLeft`, and when it is low but non zero and `ageSeconds` is at least `KeepaliveDays`, call `CodehashKeepalive` instead of waiting for expiry and calling `activateProgram`. Bound the spend with the same per contract cap and escrow checks used by the activation path, and keep `activateProgram` for the cases keepalive cannot serve, namely an expired program, a version upgrade, and a never activated program.

If keeping programs activated is not an intended goal of the contract and the activation path exists only to recover lapsed programs, this can be acknowledged as is.

CoBuilders: Acknowledged:

- current activation automation is intentionally a recovery mechanism for expired or upgrade-required programs. It does not implement proactive keepalive
- proactive keepalive is a separate lifecycle/cost feature requiring decisions around remaining-time thresholds, scheduling, escrow limits, refunds, observability, and user configuration. We will evaluate it separately rather than expand this audit remediation release. The existing recovery path remains unchanged

7.2.10 Use named mapping parameters to make the purpose of keys and values explicit

Description: Use named mapping parameters for keys and values to make their purpose explicit:

```

BiddingEscrow.sol
27:     mapping(address => uint256) private _deposits;

CacheManagerAutomation.sol
50:     mapping(address => ContractConfig[]) public userContracts;

```

CoBuilders: Fixed in commit [21fe1d2](#).

Cyfrin: Verified.

7.2.11 Remove obsolete return statement when already using named returns

Description: Remove obsolete `return` statement when already using named returns:

```

CacheManagerAutomation.sol
486:     return (userData, hasMore);

```

CoBuilders: Fixed in commit [16b88ca](#).

Cyfrin: Verified.

7.2.12 Don't initialize to default values in Solidity

Description: Solidity already initializes to default values so there's no benefit to doing so and in some instances and compiler configurations it can also result in slightly higher gas costs:

```
CacheManagerAutomation.sol
235:     for (uint256 i = 0; i < contracts.length; i++) {
279:     for (uint256 i = 0; i < contracts.length; i++) {
306:     for (uint256 i = 0; i < contracts.length; i++) {
329:     for (uint256 i = 0; i < length; i++) {
391:     for (uint256 i = 0; i < _bidRequests.length; i++) {
407:     for (uint256 i = 0; i < _activationRequests.length; i++) {
430:     for (uint256 i = 0; i < userCount; i++) {
475:     for (uint256 i = 0; i < usersToReturn; i++) {
518:     uint256 cacheUtilization = 0;
548:     uint64 decayRate = 0;
593:     for (uint256 j = 0; j < contracts.length; j++) {
713:     for (uint256 j = 0; j < contracts.length; j++) {
```

CoBuilders: Acknowledged:

- Solidity initializes these loop counters and local variables to zero by default, so explicit and implicit initialization are behaviorally equivalent
- We intentionally retain the explicit form as a readability convention. The finding has no security impact, and any potential gas benefit does not justify a broad style-only diff in this remediation release

7.3 Gas Optimization

7.3.1 CacheManagerAutomation::_shouldBid runs precompile reads before cheap registration checks

Description: CacheManagerAutomation::_shouldBid calls the arbWasmCache.codehashIsCached precompile (line 586) before it checks whether the contract is registered to the user and enabled (lines 592-596). The sibling CacheManagerAutomation::_shouldActivate does the reverse: it runs the cheap registration, autoActivate, and balance checks first and only then reads the arbWasm.programTimeLeft precompile (lines 706-730), with an in-source comment (lines 706-708) noting the ordering avoids paying for a precompile read on entries that will be skipped.

So for any batch entry that is unregistered or disabled, _shouldBid pays for the cold codehashIsCached external call before the cheap storage checks reject it. The registration and enabled checks are storage reads, which are cheaper than the cross-contract precompile call; running them first short-circuits non-bidding entries for a few thousand gas each and makes the two functions consistent.

Recommended Mitigation: Move the registration and enabled checks ahead of the codehashIsCached precompile read, matching _shouldActivate:

```
address user = bidRequest.user;
address contractAddress = bidRequest.contractAddress;

if (user == address(0) || contractAddress == address(0))
    return BidResult(false, ContractConfig(address(0), 0, false, false, 0), 0);

// Cheap storage checks first: locate the user's registered + enabled config
ContractConfig[] storage contracts = userContracts[user];
for (uint256 j = 0; j < contracts.length; j++) {
    if (contracts[j].contractAddress == contractAddress) {
        if (!contracts[j].enabled) return BidResult(false, contracts[j], 0);

        // Only now pay for the precompile read
        if (arbWasmCache.codehashIsCached(contractAddress.codehash))
            return BidResult(false, contracts[j], 0);

        uint192 minBid = cacheManager.getMinBid(contractAddress);
        // ... unchanged from here
    }
}
return BidResult(false, ContractConfig(address(0), 0, false, false, 0), 0);
```

Do not add a userBalance == 0 short-circuit: free bids (calculatedBid == 0 while the cache is below cacheThreshold) require no balance, so skipping zero-balance users would break free caching.

CoBuilders: Fixed in commit [06a45ea](#).

Cyfrin: Verified.

7.3.2 Use custom errors instead of string revert reasons

Description: Two revert sites use string literals instead of custom errors. On Solidity 0.8.x with the optimizer, a custom error (a 4-byte selector) is cheaper than a string revert, whose literal is stored in bytecode and ABI-encoded when the revert fires. The codebase already defines custom errors (InvalidAddress, InsufficientBalance, ContractNotFound, etc.), so these two are inconsistent with the existing style.

```
contracts/core/BiddingEscrow.sol
86:         revert('Amount exceeds balance');

contracts/core/CacheManagerAutomation.sol
499:         require(index < usersWithContracts.length(), 'Index out of bounds');
```

Recommended Mitigation: Replace both with custom errors, matching the pattern already used elsewhere:

```
// BiddingEscrow.sol
error AmountExceedsBalance();
if (amount > balance) revert AmountExceedsBalance();

// CacheManagerAutomation.sol
error IndexOutOfBounds();
if (index >= usersWithContracts.length()) revert IndexOutOfBounds();
```

CoBuilders: Fixed in commit [ceca9de](#) by replacing all revert strings with custom errors.

Cyfrin: Verified.

7.3.3 Pack ContractConfig address and bools into one storage slot

Description: ContractConfig orders its fields so that contractAddress and the two bools fall in separate storage slots, using 4 slots per element of the userContracts array. The two bools already pack together (they are declared adjacently), but the address does not pack with them. Reordering the bools to sit immediately after contractAddress packs address (20 bytes) + bool + bool into a single slot, reducing the struct to 3 slots, a 1-slot saving per registered contract (mostly the SSTORE writes in insertContract and updateContract).

```
contracts/interfaces/ICacheManagerAutomation.sol:8-14
struct ContractConfig {
    address contractAddress;    // slot 0 (20 bytes, 12 unused)
    uint256 maxBid;            // slot 1
    bool enabled;              // slot 2 (enabled + autoActivate already pack here)
    bool autoActivate;         // slot 2
    uint256 maxActivationCost; // slot 3
}
```

Recommended Mitigation: Move the two bools directly after contractAddress so they pack into its slot (3 slots):

```
struct ContractConfig {
    address contractAddress;    // slot 0 (20 bytes)
    bool enabled;              // slot 0
    bool autoActivate;         // slot 0 (address + 2 bools = 22 bytes)
    uint256 maxBid;            // slot 1
    uint256 maxActivationCost; // slot 2
}
```

CoBuilders: Fixed in commit [0ff4280](#).

Cyfrin: Verified.

7.3.4 Use calldata instead of memory for read-only internal struct parameters

Description: CacheManagerAutomation::_shouldBid and _shouldActivate are internal view functions called from placeBids / placeActivations with elements of calldata arrays (_bidRequests[i], _activationRequests[i]). Their struct parameters are declared memory, so each call copies the calldata element into memory. Both parameters are read-only, so declaring them calldata avoids the copy.

```
contracts/core/CacheManagerAutomation.sol
569:         BidRequest memory bidRequest,
690:         ActivationRequest memory request
```

Recommended Mitigation: Declare the read-only struct parameters as calldata:

```
function _shouldBid(BidRequest calldata bidRequest, uint256 bidIndex) internal view returns (BidResult)
↳ memory { ... }
```

```
function _shouldActivate(ActivationRequest calldata request) internal view returns (ActivationResult
↳ memory) { ... }
```

The saving is small (each struct is two address fields), and `viaIR` may already elide the copy; the change mainly aligns the annotation with the data's actual location.

CoBuilders: Fixed in commit [f264c49](#).

Cyfrin: Verified.

7.3.5 Right-size and pack owner-configurable storage parameters

Description: Most owner-configurable parameters are declared `uint256` but hold values far smaller than 2^{256} , and each currently occupies its own storage slot. Right-sizing them and grouping the small ones lets several share a slot, reducing the parameter block from 9 slots to 3. This saves storage writes at deployment and warm reads where several are read together (for example `cacheThreshold` and `horizonSeconds` in `_calculateBidAmount`).

```
contracts/core/CacheManagerAutomation.sol:32-41 (current: 9 slots)
uint256 maxContractsPerUser;           // 50
uint256 minMaxBidAmount;               // wei floor
uint256 minFundAmount;                // wei floor
uint256 maxUserFunds;                 // wei (1 ether default)
uint256 maxBidsPerIteration;          // 50
uint256 maxUsersPerPage;              // 100
uint256 cacheThreshold;               // <= 100
uint256 horizonSeconds;               // seconds (30 days default)
uint256 maxActivationsPerIteration;   // 5
```

Recommended Mitigation: Declare the parameters grouped by size so they pack (3 slots):

```
// slot 1: 6 small parameters
uint40 maxContractsPerUser;
uint40 maxBidsPerIteration;
uint40 maxUsersPerPage;
uint40 maxActivationsPerIteration;
uint32 cacheThreshold;
uint64 horizonSeconds;
// slots 2-3: wei values (uint128 ~= 3.4e38 wei, far above total ETH supply)
uint128 maxUserFunds;
uint128 minFundAmount;
uint128 minMaxBidAmount;
```

Before applying, note the usage check:

- Every current usage except one is a comparison or an assignment to a `uint256`, which Solidity widens automatically and is safe.
- The single arithmetic site is `_calculateBidAmount` at line 558, `uint256(decayRate) * horizonSeconds`. This is safe only because `decayRate` is widened to `uint256`, so the multiplication runs in `uint256`. Keep both operands widened, i.e. `uint256(decayRate) * uint256(horizonSeconds)`. Do NOT multiply the narrow types directly (`decayRate * horizonSeconds`): that would compute in the narrow type and revert on overflow, bricking `placeBids`. With `horizonSeconds` as `uint64` the product is bounded below 2^{128} , so it can no longer overflow `decayValue`.
- Setters and their interface declarations: validate each input fits its new type (the existing bounds mostly suffice, e.g. `cacheThreshold <= 100`) and cast on store, or narrow the setter parameters. The public getters narrow to the new types (an ABI change, acceptable on a fresh deployment).

CoBuilders: Acknowledged & deferred:

- we implemented and benchmarked a candidate that narrows and packs the owner-configurable values. It reduced `placeBids` by approximately 3.8k gas but increased deployment by 13,032 gas, insertion by 174 gas, setters by approximately 69-133 gas, and runtime bytecode by 776 bytes

- the preferred implementation changes public getter widths, requiring coordinated ABI/client migration. An ABI-preserving alternative retained similar bidding savings but added validation and implementation complexity
- because this is a gas optimization rather than a correctness or security fix, we retain the current layout for this release. The experiment is absent from the remediation branch and preserved for dedicated follow-up work

7.3.6 Use `msg.sender` inside `onlyOwner` functions instead of `owner()` to optimize away unnecessary storage read

Description: Use `msg.sender` inside `onlyOwner` functions instead of `owner` to optimize away unnecessary storage read:

```
BiddingEscrow.sol
91: payable(owner()).sendValue(amount);
```

CoBuilders: Fixed in commit [d494949](#).

Cyfrin: Verified.

7.3.7 Use `ReentrancyGuardTransient` for faster `nonReentrant` modifiers

Description: Use `ReentrancyGuardTransient` for faster `nonReentrant` modifiers:

```
CacheManagerAutomation.sol
7:import {ReentrancyGuard} from '@openzeppelin/contracts/security/ReentrancyGuard.sol';
19: ReentrancyGuard
```

CoBuilders: Acknowledged & deferred:

- `ReentrancyGuardTransient` requires OpenZeppelin 5.1+ and a Cancun-compatible target with TLOAD/TSTORE. This release intentionally uses OpenZeppelin 4.9.6 and targets Paris
- applying it now would combine a major dependency migration, global EVM-target change, cross-network transient-storage validation, and broader compatibility review for a gas-only optimization. The existing storage-backed `ReentrancyGuard` remains active. We will evaluate this in a dedicated OpenZeppelin 5+/Cancun migration

7.3.8 Emit events first to optimize away old values only used for event emission

Description: Emit events first to optimize away old values only used for event emission, eg:

```
function setMaxContractsPerUser(
    uint256 _maxContractsPerUser
) external onlyOwner {
    require(
        _maxContractsPerUser > 0,
        'Max contracts per user must be greater than 0'
    );
-   uint256 oldValue = maxContractsPerUser;
+   emit MaxContractsPerUserUpdated(maxContractsPerUser, _maxContractsPerUser);
    maxContractsPerUser = _maxContractsPerUser;
-   emit MaxContractsPerUserUpdated(oldValue, _maxContractsPerUser);
}
```

Also affects:

```
97: function setMaxContractsPerUser(
111: function setMinMaxBidAmount(uint256 _minMaxBidAmount) external onlyOwner {
123: function setMinFundAmount(uint256 _minFundAmount) external onlyOwner {
132: function setMaxUserFunds(uint256 _maxUserFunds) external onlyOwner {
```

```

141:     function setMaxBidsPerIteration(
155:     function setMaxUsersPerPage(uint256 _maxUsersPerPage) external onlyOwner {
167:     function setCacheThreshold(uint256 _cacheThreshold) external onlyOwner {
176:     function setHorizonSeconds(uint256 _horizonSeconds) external onlyOwner {
185:     function setBidIncrement(uint192 _bidIncrement) external onlyOwner {
194:     function setMaxActivationsPerIteration(

```

CoBuilders: Acknowledged:

- we retain the explicit flow of reading the old value, updating storage, and then emitting the old/new event. This effects-before-event sequence is straightforward to review
- these setters are rare owner-only operations expected to execute through a Safe rather than a transactional hot path. The potential gas saving is negligible in this context and does not justify changing all setter flows

7.3.9 Use named return variables when this can optimize away a local variable

Description: Use named return variables when this can optimize away a local variable, eg:

```

-   function getContracts() external view returns (UserContractsData[] memory) {
+   function getContracts() external view returns (UserContractsData[] memory allUserContracts) {
    uint256 userCount = usersWithContracts.length();
-   UserContractsData[] memory allUserContracts = new UserContractsData[] (
+   allUserContracts = new UserContractsData[] (
        userCount
    );

    for (uint256 i = 0; i < userCount; i++) {
        address user = usersWithContracts.at(i);
        allUserContracts[i] = UserContractsData({
            user: user,
            contracts: userContracts[user]
        });
    }

-   return allUserContracts;
}

```

CoBuilders: Fixed in commit [a8b7bbe](#).

Cyfrin: Verified.

7.3.10 Cache identical storage reads

Description: In EVM reading from storage is expensive, hence identical storage reads should be cached if there is no possibility of the value changing prior to the subsequent reads:

```

CacheManagerAutomation.sol
// cache contracts.length in insertContract, updateContract, removeContract, _shouldBid, _shouldActivate
// interestingly removeAllContracts caches it but the other functions dont
235:     for (uint256 i = 0; i < contracts.length; i++) {
242:     if (contracts.length == 0) {
279:     for (uint256 i = 0; i < contracts.length; i++) {
306:     for (uint256 i = 0; i < contracts.length; i++) {
308:         contracts[i] = contracts[contracts.length - 1];
// here the value has changed due to pop() but this is known so just deduct 1 from the cached value
312:     if (contracts.length == 0) {
593:     for (uint256 j = 0; j < contracts.length; j++) {
713:     for (uint256 j = 0; j < contracts.length; j++) {

```

CoBuilders: Fixed in commit [3ce768a](#).

Cyfrin: Verified.